

Invoice & Receipt Collector — Team Brief

Date: August 29, 2026

Design decisions so far, for group review before we build. Please read and flag anything you'd change before we lock this in.

The Problem

We're building a GenAI expense tracker for a **grad student on a budget** — someone who wants to understand their monthly spending without manually logging every purchase, and who wants to just *ask* questions about their spending in plain English instead of digging through numbers.

How It Works (User's Perspective)

1. User uploads a **synthetic receipt image** (not a real photo — this keeps our live demo reliable and avoids messy OCR problems) or types a quick note like “\$4 on chicken rice today.”
2. An AI reads it and shows the user an editable table row: date, merchant, amount, description. **The user confirms or corrects it** before it's saved — we don't auto-save AI guesses.
3. The expense gets automatically **categorized** by matching it against a master lookup table we build in Excel (e.g., “Trader Joe's” → Groceries). If nothing matches, it goes into an **Uncategorized** bucket for the user to sort later. *(We're intentionally not using AI for categorization — a table lookup is more consistent and easier to explain/evaluate than letting the AI guess.)*
4. Three tabs from there:
 - **Category tab** — browse and manage logged expenses
 - **Query tab** — ask natural language questions (“what did I spend most on?”, “how much would I save cutting \$10/week off groceries?”)

- **Summary tab** — a written monthly summary + graphs (graphs are built with Python reading our data table directly, not drawn by the AI)

The Two AI Modules (for Stage 3)

	Module 1: Extraction	Module 2: Query & Summary
Job	Turn a receipt image or text note into structured data	Answer natural language questions about spending
Model	Qwen-VL (vision)	DeepSeek Chat (text + tool-calling)
Key design choice	Output strict JSON only, never guess a field it can't read	Never do math itself — always calls a calculator tool for arithmetic

Both run via **OpenRouter**, so we're not tied to Gemini Canvas — we can wire these into whatever front end we build.

Why Two Different Models?

Vision extraction and natural-language reasoning are genuinely different jobs, so we picked a model suited to each rather than using one model for everything:

- **Qwen-VL** — actual vision-language model, good at reading images into structured output
- **DeepSeek Chat** — cheap, strong at reasoning/tool-calling, no vision needed for this module

Evaluation Plan (Stage 6)

We're doing **human evaluation**, not AI-judging-AI. Process:

1. Build **3 versions** of the system:
 - **(A) Minimal** — bare LLM, no prompt engineering, no tools
 - **(B) Simplified** — has a prompt and sees the data, but no smart retrieval and no calculator tool (LLM does its own math — likely where we'll see errors)
 - **(C) Full system** — our actual design: retrieval + tool-calling
2. Run **20 test cases** (10 for extraction, 10 for query/summary — covering normal cases, ambiguous input, missing info, and one deliberate prompt injection test) through all three versions

3. A human on the team scores each output 1–5 on four criteria:

Criterion	What it means
Grounding	Did it only use real data, or did it make something up?
Correctness	Is the math and categorization actually right?
Readability	Is the answer clear and easy to read?
Safety	Does it stay in scope and resist manipulation (e.g. an injected instruction hidden in a transaction description)?

We'll also separately compare the system to a **hand-written ledger** — how much time it saves and whether it catches errors a manual process would miss. That's a different kind of evidence (practical usefulness), so it sits alongside the scoring table, not inside it.

Open Items for the Team

☐

Finalize 2–4 success criteria for Stage 1

☐

Confirm exact Qwen-VL and DeepSeek Chat model strings are live and working correctly on OpenRouter (image input + tool-calling support)

☐

Build the Excel category master table

☐

Assign who scores which test cases (and whether we blind the variant labels while scoring)

☐

Decide which AI builder tool we'll use to implement the UI (not Canvas)

Full technical spec (for whoever's driving the build) is in `PROJECT_SPEC.md`.